

A. Project Overview

Goal: To identify how collaboration patterns among actors and directors on Netflix reveal connectivity between figures, influential individuals, and the most tightly connected creative communities across genres.

Dataset

- Source: [Netflix Movies and TV Shows \(Kaggle\)](#)
- Size: 3.4 MB (8807 data points)

B. Data Processing

The dataset was loaded into Rust using the “csv” and “serde” crates, which allowed each row in the Netflix CSV file to be deserialized into a custom ‘NetflixRecord’ struct. This struct includes optional fields for cast, director, and genre information. During preprocessing, cast and director names were split by commas and trimmed to ensure consistent formatting. Genres were also extracted into a vector of strings for easier frequency analysis. These cleaned and structured records were then used to construct an undirected graph, where each node represents a person, and an edge is created between any two individuals who collaborated on the same title.

C. Code Structure

Modules

- **main.rs:** Orchestrates execution by loading data, building graphs, and running analysis.
- **parse.rs:** In charge of reading the CSV file and turning each row into a usable format. It also helps clean up the data so names and genres are consistent.
- **graph_builder.rs:** Constructs an undirected graph from the Netflix dataset, connecting collaborators.
- **analysis.rs:** Runs graph analysis, including average path, clustering, centrality, and community detection.

This organization reflects clean architectural practices and ensures each module has a focused role in the data → graph → analysis process.

Key Structs and Enums:

- NetflixRecord represents a single row from the dataset, storing optional cast, director, and genre data. It provides helper methods to check if a person is in a record (has_person) and to extract genres (genres). Both methods return cleaned, usable data formats for further analysis.

- Role is an enum with two variants: Cast and Director. It's used to clearly indicate which role a person is being checked for in a title, improving code readability and reducing ambiguity.

Key Function:

- `load_data(path: &str)` reads the CSV file and returns a vector of `NetflixRecord` structs by deserializing each row and skipping invalid ones. This function ensures the data is ready for graph construction.
- `build_graph(records: &[NetflixRecord])` creates an undirected graph where each node is a person, and edges connect people who worked on the same title. It uses a `HashMap` to avoid duplicate nodes and iterates through all people combinations to add edges.
- `average_path_length(graph)` estimates the average number of steps between people in the graph (i.e. six degrees of separation). It does this by running a BFS from up to 100 random nodes and averaging the total distances found.
- `clustering_info(graph)` calculates the global clustering coefficient by counting how often a person's neighbors are also connected to each other (triangles), compared to how often they could be (triplets).
- `centrality_info(graph, top_n)` ranks people by how many others they are directly connected to and returns the top N. It builds a degree map from node connections and sorts it in descending order.
- `local_clustering(graph, people)` computes the clustering coefficient for a list of people by measuring how interconnected their neighbors are, which helps evaluate their local influence in the network.
- `detect_communities(graph, records)` finds connected groups of people (communities) using BFS, then analyzes the genres that appear most often in each group. It prints out a summary of the top five communities and their most common genres.

Main Workflow

The program starts in `main.rs`, which first calls `load_data()` from `parse.rs` to read and clean the dataset. Then, it passes the cleaned data to `build_graph()` from `graph_builder.rs`, which constructs the collaboration network. Once the graph is built, several analysis functions from `analysis.rs` are called: `average_path_length()` measures overall connectedness, `clustering_info()` checks how tightly the network is grouped, and `centrality_info()` finds the most connected people. The names from that centrality result are passed to `local_clustering()` to see how clustered their neighbors are. Finally, `detect_communities()` identifies groups of closely connected individuals and analyzes what genres they tend to work in. Each module hands off its result to the next step, creating a smooth pipeline from raw data to meaningful insights.

D. Tests

```
$ cargo test
Compiling finalproject v0.1.0 (/opt/app-root/src/finalproject)
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.71s
Running unittests src/main.rs (target/debug/deps/finalproject-a5d571df61494154)

running 5 tests
test analysis::tests::test_centrality ... ok
test analysis::tests::test_local_clustering ... ok
test analysis::tests::test_detect_communities ... ok
test parse::tests::test_is_person_cast ... ok
test graph_builder::tests::test_build_graph ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

test_centrality

What it checks: Ensures that the person with the most connections is correctly identified at the top of the centrality ranking.

Why it matters: Centrality is one of the core metrics in this project—this confirms the ranking is computed correctly.

test_local_clustering

What it checks: Runs the local clustering function on a small triangle graph to confirm that the function calculates and prints valid results.

Why it matters: Helps verify that the local clustering logic correctly reflects real-world connectivity in small networks.

test_detect_communities

What it checks: Uses a graph with two clearly separate communities and checks that the function runs successfully and associates genres correctly.

Why it matters: Ensures community detection works as expected and that genre analysis within communities is handled without error.

test_is_person_cast

What it checks: Confirms that a person listed in the cast is correctly detected by the `has_person` method, and that someone not in the cast is not falsely matched.

Why it matters: Ensures accuracy when building connections in the graph based on cast and director roles.

test_build_graph

What it checks: Builds a small example graph and verifies that the correct number of nodes and edges are created from the input records.

Why it matters: Validates that the graph-building logic properly captures collaborations from the dataset.

E. Results

```
$ cargo run --release
  Compiling finalproject v0.1.0 (/opt/app-root/src/finalproject)
  Finished `release` profile [optimized] target(s) in 1.72s
  Running `target/release/finalproject`
Average path length (six degrees): 5.05
Clustering coefficient: 0.3715
Top 5 most connected people:
Anupam Kher (301 connections)
Samuel L. Jackson (256 connections)
Fred Tatasciore (242 connections)
Shah Rukh Khan (234 connections)
Takahiro Sakurai (233 connections)

Local clustering coefficients:
Anupam Kher: 0.0689
Samuel L. Jackson: 0.0597
Fred Tatasciore: 0.1017
Shah Rukh Khan: 0.0860
Takahiro Sakurai: 0.1746

Community detection:
Detected 1094 communities.

Community 1 (36878 people): ["Ama Qamata", "Khosi Ngema", "Gail Mabalan", "Thabang Molaba", "Noxee Maqashalala"]
Common genre: Dramas (25632 titles)

Community 2 (34 people): ["Jeferson De", "Juan Paiva", "Mariana Nunes", "Giulia Gayoso", "Bruno Peixoto"]
Common genre: International Movies (25 titles)
Common genre: Dramas (25 titles)

Community 3 (26 people): ["Jeremy Grant", "Kurt Sorge", "Tom Van Steenberg", "Cameron Zink", "Graham Agassiz"]
Common genre: International Movies (28 titles)
Common genre: Sports Movies (28 titles)

Community 4 (23 people): ["Guri Alfi", "Shai Avivi", "Yehuda Levi", "Yehoram Gaon", "Liron Vaisman"]
Common genre: International TV Shows (24 titles)
Common genre: TV Dramas (24 titles)

Community 5 (22 people): ["Alex Woo", "Stanley Moore", "Maisie Benson", "Anna Chambers", "Paul Killam"]
Common genre: Children & Family Movies (52 titles)
```

The results show that the average path length in the Netflix collaboration network is approximately 5.05, which supports the “six degrees of separation” theory. This suggests that, on average, any two people in the dataset are only about five steps apart through shared film or TV projects.

The clustering coefficient of 0.3715 indicates a moderately high tendency for collaborators of one person to also work with each other. This is typical in creative industries where actors and directors often work in repeated groups or genre circles.

The top five most connected people include highly prolific actors like Anupam Kher and Samuel L. Jackson, confirming that the centrality analysis effectively identifies major figures in the network. Their relatively low local clustering scores suggest they act as bridges between otherwise separate groups.

Community detection uncovered over 1,000 distinct collaboration groups. The largest community, with over 36,000 members, is heavily centered around the “Dramas” genre. Smaller communities also show genre cohesion, such as one combining “International Movies” and “Dramas”, and another focused on “Children & Family Movies.” This indicates that genre plays

a key role in shaping collaborative clusters on Netflix, and that people tend to work repeatedly within the same content categories.

Overall, the analysis successfully reveals meaningful patterns about how people collaborate in the Netflix ecosystem—highlighting influential individuals, tightly connected genre communities, and overall network structure.

F. Usage Instructions

To run the project, the `netflix_titles.csv` dataset should be placed in the root directory alongside `Cargo.toml`. The code is compiled using `cargo build --release` and runs the program with `cargo run --release`.

There are no command-line arguments or user interaction required—everything runs automatically. Once executed, the program loads and parses the dataset, builds the collaboration graph, and performs all the analyses: six degrees of separation, clustering coefficient, centrality, local clustering, and community detection with genre analysis. The results are printed directly to the terminal.

On my machine, the full analysis runs in about 1–2 seconds. Runtime may vary slightly depending on system speed, but it completes quickly even with the full `netflix_titles.csv` dataset.